MODUL PEMBELAJARAN

Pemrograman Web Statis

JavaScript untuk Web: DOM Selection

Kelas: X (Fase E) | **Jurusan:** Rekayasa Perangkat Lunak (RPL)

Alokasi waktu: 12 JP (4 pertemuan × 3 JP @ 45 menit)

Nama Siswa	Kelas	Tanggal
Naufal Fevin Ardiansyah Fedrik	XRPL.4	07/10/2026

DAFTAR ISI

1. Identitas dan Tujuan Pembelajaran
2. Petunjuk Penggunaan Modul
3. Pretest
4. Bagian 1: Mengenal JavaScript dan DOM
5. Bagian 2: Memilih Elemen dengan `getElementById`
6. Bagian 3: Memilih Elemen dengan `getElementsByClassName` dan `getElementsByTagName`
7. Bagian 4: Memilih Elemen dengan `querySelector` dan `querySelectorAll`
8. Bagian 5: Setelah Dipilih, Lalu Apa? (Mengubah Teks dan Gaya)
9. Kesalahan Umum dan Cara Memperbaikinya
10. Rangkuman dan Glosarium
11. Refleksi
12. Posttest
13. Tugas Praktikum
14. Lampiran untuk Guru: Kunci Jawaban dan Rubrik

1. IDENTITAS DAN TUJUAN PEMBELAJARAN

Capaian yang Diharapkan

Siswa mampu membuat halaman web yang dapat dikendalikan menggunakan JavaScript dengan cara memilih (menyeleksi) elemen HTML melalui DOM.

Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu diharapkan mampu:

1. Menjelaskan pengertian JavaScript dan fungsinya pada web.
2. Menjelaskan pengertian DOM (*Document Object Model*) beserta struktur pohonnya.
3. Menghubungkan file JavaScript ke halaman HTML dengan benar.
4. Memilih elemen HTML menggunakan `getElementById`, `getElementsByClassName`, dan `getElementsByTagName`.
5. Memilih elemen HTML menggunakan `querySelector` dan `querySelectorAll`.
6. Membedakan hasil dari setiap metode seleksi (satu elemen atau banyak elemen).
7. Mengubah teks dan tampilan elemen yang sudah dipilih.
8. Menemukan dan memperbaiki kesalahan umum saat menyeleksi elemen.

Prasyarat

- Sudah mengenal struktur dasar HTML (`<html>`, `<head>`, `<body>`).
- Sudah mengenal tag dasar: `<h1>`, `<p>`, `<ul>`, `<li>`, `<div>`, `<button>`.
- Sudah mengenal atribut `id` dan `class`.
- Sudah mengenal selector CSS dasar.

Alat dan Bahan

- Laptop/PC
- Browser (Google Chrome / Microsoft Edge / Firefox)
- Teks editor (Visual Studio Code, disarankan)
- Folder kerja: buat folder bernama `modul-dom` di komputermu

Rencana Pertemuan

Pertemuan	Kegiatan
1	Pretest, Bagian 1 (JavaScript dan DOM), Praktikum 1
2	Bagian 2 dan 3 (`getElementById`, `ClassName`, `TagName`), Praktikum 2 dan 3
3	Bagian 4 dan 5 (`querySelector`, mengubah elemen), Praktikum 4 dan 5
4	Tugas praktikum, Posttest, Refleksi

2. PETUNJUK PENGGUNAAN MODUL

1. Kerjakan **Pretest** terlebih dahulu **tanpa membuka materi**. Jawaban salah itu tidak apa-apa, karena pretest hanya untuk mengetahui kemampuan awalmu.
2. Baca materi secara berurutan. Jangan melompat ke bagian berikutnya sebelum paham.
3. **Ketik sendiri** semua kode di praktikum, jangan hanya *copy-paste*. Mengetik sendiri melatih mata dan tanganmu menemukan kesalahan.
4. Setiap praktikum diakhiri **Tantangan Mandiri**. Kerjakan tanpa melihat contoh.
5. Jika kodemu tidak jalan, buka **Console** browser (tekan `F12`, pilih tab *Console*). Pesan error berwarna merah adalah petunjuk utamamu.
6. Isi **Refleksi** dengan jujur. Tidak ada nilai untuk refleksi yang "benar", yang dinilai adalah kejujuranmu.
7. Kerjakan **Posttest** dan **Tugas Praktikum** secara mandiri.

3. PRETEST

Petunjuk: Pilih satu jawaban yang paling tepat. Waktu: 10 menit.

1. Pada pengembangan web, JavaScript digunakan untuk ...

A. menentukan struktur isi halaman
B. membuat halaman web menjadi interaktif
C. mengatur warna dan jenis huruf saja
D. menyimpan data dalam jumlah besar

2. Tag HTML yang digunakan untuk menuliskan atau menghubungkan kode JavaScript adalah ...

A. `<script>`
B. `<js>`
C. `<style>`
D. `<link>`

3. DOM adalah singkatan dari ...

A. Data Object Model
B. Display Output Mode
C. Document Object Model
D. Digital Operation Method

4. Perintah `console.log("Halo")` berfungsi untuk ...

A. membuat tombol bertuliskan Halo
B. mengubah judul halaman menjadi Halo
C. menghapus tulisan Halo dari halaman
D. menampilkan tulisan Halo di Console browser

5. Aturan penulisan atribut `id` yang benar dalam satu halaman HTML adalah ...

A. boleh sama pada banyak elemen
B. harus unik (tidak boleh kembar)
C. harus berupa angka
D. boleh dikosongkan

6. Selector CSS untuk memilih elemen dengan `class="kartu"` adalah ...

A. `.kartu`
B. `#kartu`
C. `kartu#`
D. `*kartu`

7. Selector CSS untuk memilih elemen dengan `id="menu"` adalah ...

A. `.menu`
B. `&menu`
C. `@menu`
D. `#menu`

8. Ekstensi file untuk menyimpan kode JavaScript adalah ...

A. `.java`
B. `.html`
C. `.js`
D. `.script`

9. Pada kode `<p>Halo</p>`, keseluruhan bagian tersebut disebut ...

A. atribut
B. elemen
C. properti
D. komentar

10. Cara menghubungkan file `script.js` ke halaman HTML yang benar adalah ...

A. `<script src="script.js"></script>`
B. `<link src="script.js">`
C. `<js href="script.js"></js>`
D. `<style src="script.js"></style>`

> Skor pretest = (jumlah benar × 10). Catat skormu: _____

4. BAGIAN 1: MENGENAL JAVASCRIPT DAN DOM

4.1 Peran JavaScript pada Web

Sebuah halaman web dibangun oleh tiga "pemain" utama. Bayangkan sebuah **rumah**:

Teknologi	Peran	Analogi rumah
HTML	Struktur dan isi halaman	Tembok, pintu, jendela (kerangka rumah)
CSS	Tampilan dan gaya	Cat, dekorasi, warna
JavaScript	Perilaku dan interaksi	Listrik: lampu bisa nyala, pintu otomatis terbuka

Tanpa JavaScript, halaman web hanya bisa **dilihat**. Dengan JavaScript, halaman web bisa **bereaksi**: tombol bisa diklik lalu sesuatu berubah, teks bisa berganti, form bisa dicek otomatis, dan sebagainya.

4.2 Apa Itu DOM?

DOM (Document Object Model) adalah cara browser "memahami" halaman HTML kita. Saat browser membuka file HTML, browser mengubahnya menjadi **struktur pohon** yang berisi objek-objek. Lewat DOM, JavaScript bisa **menemukan, membaca, dan mengubah** bagian mana pun dari halaman.

Perhatikan HTML berikut:

```
<html>
  <body>
    <h1 id="judul">Halo</h1>
    <p>Belajar DOM</p>
  </body>
</html>
```

Browser mengubahnya menjadi pohon seperti ini:

```
document
   └── html
        └── body
             ├── h1 (id="judul")   →  "Halo"
             └── p                 →  "Belajar DOM"
```

Istilah penting dalam pohon DOM:

- **Node**: setiap "titik" di pohon.
- **Elemen**: node yang berupa tag HTML (`html`, `body`, `h1`, `p`).
- **Parent (induk)**: elemen yang membungkus elemen lain. Contoh: `body` adalah parent dari `h1`.
- **Child (anak)**: elemen yang berada di dalam elemen lain. Contoh: `h1` adalah child dari `body`.
- **Sibling (saudara)**: elemen yang sejajar. Contoh: `h1` dan `p`.

> **Analogi:** DOM seperti **silsilah keluarga**. Untuk menemukan seseorang, kamu bisa memanggil namanya, memanggil "semua anggota keluarga A", atau menyebut posisinya. Dalam JavaScript, kegiatan "menemukan seseorang" ini disebut **DOM Selection (menyeleksi elemen)**.

4.3 Apa Itu DOM Selection?

DOM Selection adalah proses **memilih elemen HTML** agar bisa kita kendalikan lewat JavaScript. Ini adalah langkah **pertama** sebelum kita bisa mengubah apa pun di halaman.

Alurnya selalu begini:

```
1. PILIH elemen   →   2. UBAH / BACA elemen
```

Kamu tidak bisa mengubah sesuatu yang belum kamu pilih.

Ada lima cara utama memilih elemen yang akan kita pelajari:

No	Metode	Memilih berdasarkan	Hasil
1	`getElementById()`	id	1 elemen
2	`getElementsByClassName()`	class	banyak elemen
3	`getElementsByTagName()`	nama tag	banyak elemen
4	`querySelector()`	selector CSS	1 elemen (yang pertama)
5	`querySelectorAll()`	selector CSS	banyak elemen

4.4 Tiga Cara Menyisipkan JavaScript ke HTML

a. Inline (langsung di atribut elemen), tidak disarankan untuk kode panjang:

```
<button onclick="alert('Halo!')">Klik</button>
```

b. Internal (di dalam tag `<script>` pada file HTML):

```
<script>
  console.log("Halo dari internal");
</script>
```

c. Eksternal (file `.js` terpisah), **cara terbaik dan yang kita pakai di modul ini**:

```
<script src="script.js"></script>
```

Mengapa eksternal lebih baik? Kode lebih rapi, mudah dicari, dan bisa dipakai ulang di banyak halaman.

4.5 Letak Tag `<script>` Sangat Penting!

Browser membaca HTML dari **atas ke bawah**. Jika `<script>` diletakkan **sebelum** elemen yang dicari, JavaScript akan mencari elemen yang **belum ada**, hasilnya `null` atau error.

```
<!-- ❌ SALAH: script di atas, elemen belum dibuat -->
<head>
  <script src="script.js"></script>
</head>
<body>
  <h1 id="judul">Halo</h1>
</body>

<!-- ✅ BENAR: script di akhir body, elemen sudah ada -->
<body>
  <h1 id="judul">Halo</h1>

  <script src="script.js"></script>
</body>
```

Aturan praktis: letakkan `<script src="...">` **tepat sebelum** `</body>`.

> **Analogi:** bayangkan guru mengabsen siswa **sebelum** siswa masuk kelas. Kelas masih kosong, jadi tidak ada yang menjawab. Itulah yang terjadi jika `<script>` dijalankan sebelum elemen HTML dibuat: JavaScript memanggil elemen yang belum ada, hasilnya `null`. Maka absen dilakukan **setelah** semua siswa duduk, yaitu script di akhir `<body>`.

4.6 Mengenal Console

Console adalah "layar catatan" milik browser untuk programmer. Fungsinya menampilkan hasil `console.log()` dan pesan error.

> **Analogi:** Console seperti **papan skor dan buku catatan di bengkel**. Mekanik (programmer) menuliskan hasil pengecekan dan melihat tanda bahaya (error berwarna merah) di sana, sementara pelanggan (pengguna web) tidak melihatnya.

Cara membuka: tekan **F12** (atau klik kanan → *Inspect*) lalu pilih tab **Console**.

```
console.log("Halo, Console!");    // menampilkan teks
console.log(2 + 3);               // menampilkan hasil hitungan: 5
```

Tanda `//` adalah **komentar**, yaitu catatan untuk manusia yang tidak dijalankan komputer.

🧪 PRAKTIKUM 1: Menghubungkan JavaScript dan Menjelajahi DOM

Tujuan: Berhasil menghubungkan file JS dan melihat objek DOM di Console.

Langkah 1. Di folder `modul-dom`, buat folder baru `praktikum1`. Di dalamnya buat dua file: `index.html` dan `script.js`.

Langkah 2. Isi `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum 1 - Mengenal DOM</title>
</head>
<body>
  <h1 id="judul">Halo, Dunia Web!</h1>
  <p>Ini paragraf pertama.</p>
  <p>Ini paragraf kedua.</p>

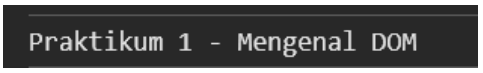
  <script src="script.js"></script>
</body>
</html>
```

Langkah 3. Isi `script.js`:

```
console.log("JavaScript sudah terhubung!");
console.log(document.title);
console.log(document.body);
console.log(document);
```

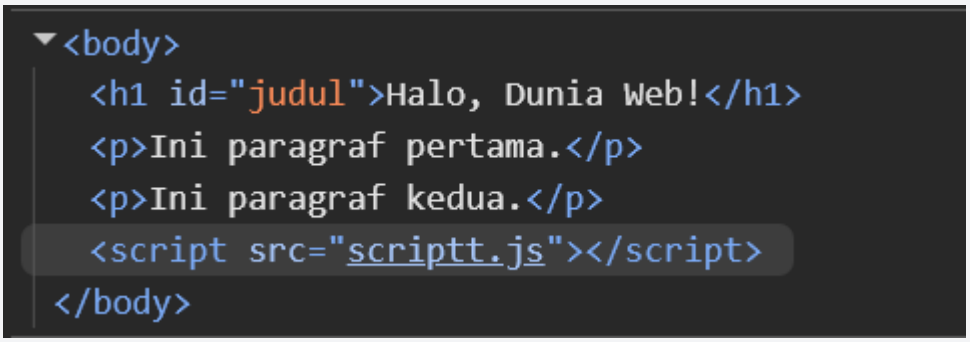
Langkah 4. Buka `index.html` di browser. Tekan `F12`, buka tab **Console**.

Langkah 5. Jawab pertanyaan pengamatan:

Perintah	Apa yang muncul di Console?
`console.log("JavaScript sudah terhubung!")`	
`console.log(document.title)`	
`console.log(document.body)`	

Langkah 6. Di Console, klik tanda segitiga ▸ di samping `<body>` untuk membuka isinya. Gambarkan pohon DOM halamanmu di kotak berikut:

```
(gambar pohon DOM di sini)
```



🎯 **Tantangan Mandiri 1:** Ubah `<title>` menjadi namamu, lalu tampilkan di Console dengan `console.log(document.title)`. Apakah hasilnya berubah?

Pertanyaan Diskusi:

1. Apa yang terjadi jika kamu memindahkan `<script src="script.js"></script>` ke dalam `<head>`? (Untuk praktikum 1 mungkin belum terlihat bedanya, tapi di praktikum berikutnya akan terasa.)
2. Apa perbedaan HTML yang kamu tulis dengan DOM yang dilihat browser?

Refleksi singkat Bagian 1: Satu hal baru yang aku pelajari: Objek document di JavaScript memungkinkan kita untuk mengakses dan memanipulasi seluruh elemen halaman HTML

5. BAGIAN 2: MEMILIH ELEMEN DENGAN `getElementById`

5.1 Konsep

`getElementById()` memilih **satu elemen** berdasarkan nilai atribut `id`-nya.

> **Analogi:** `id` seperti **NIS (Nomor Induk Siswa)**. Setiap siswa punya NIS yang berbeda. Kalau guru memanggil "NIS 12345", pasti hanya satu orang yang berdiri. Karena itulah `id` harus **unik**.

5.2 Sintaks

```
document.getElementById("namaId");
```

Bagian-bagiannya:

- `document` : seluruh halaman web
- `getElementById` : perintah "cari elemen berdasarkan id"
- `"namaId"` : id yang dicari (ditulis dalam tanda kutip, **tanpa tanda #**)

Biasanya hasilnya disimpan dalam **variabel** agar mudah dipakai ulang:

```
const judul = document.getElementById("judul");
```

`const` artinya "membuat wadah bernama `judul` yang isinya tidak diganti-ganti ke benda lain". Wadah itu sekarang berisi **elemen** yang kita cari.

> **Analogi:** variabel seperti **kotak penyimpanan yang diberi label**. Label `judul` ditempel pada kotak, dan isi kotaknya adalah elemen `<h1>` yang kita temukan. Kapan pun kita butuh elemen itu, cukup sebut labelnya.

5.3 Hasil Jika Tidak Ditemukan

Jika id tidak ada di halaman, hasilnya adalah **`null`** (artinya "tidak ada apa-apa"). Mencoba mengubah `null` akan menyebabkan error:

> **Analogi:** memanggil NIS yang tidak terdaftar di sekolah. Tidak ada satu pun siswa yang maju, itulah `null`. Kalau kita lalu menyuruh "siswa" itu mengerjakan sesuatu, tentu terjadi error karena orangnya memang tidak ada.

```
Uncaught TypeError: Cannot read properties of null
```

Kalau melihat error ini, pertama-tama **periksa ejaan id** dan **letak tag script**.

5.4 Perhatikan Huruf Besar dan Kecil

JavaScript membedakan huruf besar dan kecil (*case-sensitive*):

```
document.getElementById("judul");    // ✅ benar
document.getElementById("Judul");    // ❌ beda dengan "judul"
document.getElementbyId("judul");    // ❌ penulisan "getElementById" salah (b
harus B)
```

🧪 PRAKTIKUM 2: Memilih Elemen dengan `getElementById`

Langkah 1. Buat folder `praktikum2` berisi `index.html` dan `script.js`.

Langkah 2. Isi `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum 2 - getElementById</title>
</head>
<body>
  <h1 id="judul">Selamat Datang</h1>
  <p id="sapaan">Ini adalah teks awal.</p>

  <script src="script.js"></script>
</body>
</html>
```

Langkah 3. Isi `script.js`:

```
// 1. Memilih elemen
const judul = document.getElementById("judul");
const sapaan = document.getElementById("sapaan");

// 2. Melihat elemen di Console
console.log(judul);
console.log(sapaan);

// 3. Mengubah isi teks
judul.textContent = "Judul Sudah Diubah!";

// 4. Mengubah warna
judul.style.color = "crimson";

// 5. Mengubah isi dengan tag HTML
sapaan.innerHTML = "Halo, saya sedang <b>belajar DOM</b>!";

// 6. Mencoba id yang tidak ada
const hantu = document.getElementById("tidakada");
console.log(hantu);
```

Langkah 4. Jalankan di browser dan jawab:

Pertanyaan	Jawabanmu
Apa tampilan judul sekarang?	Judul Sudah Diubah!
Apa tampilan paragraf sekarang?	`// 5. Mengubah isi dengan tag HTML` `sapaan.innerHTML = "Halo, saya sedang <b>belajar DOM</b>!";`
Apa yang muncul di Console untuk `hantu`?	`// 6. Mencoba id yang tidak ada` `const hantu = document.getElementById("tidakada");` `console.log(hantu);`

Langkah 5 (Eksperimen Error). Tambahkan baris ini di akhir `script.js`, lalu amati Console:

```
hantu.textContent = "Halo";
```

Tulis pesan error yang muncul: Cannot set properties of null (setting 'textContent')

Mengapa error itu muncul? Error terjadi karena fungsi `document.getElementById("tidakada")` tidak menemukan elemen dengan ID tersebut di HTML, sehingga variabel `hantu` bernilai `null`. Saat kode

mencoba mengakses properti `.textContent` dari variabel yang bernilai `null`, JavaScript melempar `TypeError`.

Hapus baris tersebut setelah selesai.

🎯 **Tantangan Mandiri 2:**

1. Tambahkan elemen `<p id="kelas">` di HTML, isi dengan "X RPL 1".
2. Dengan JavaScript, ubah isinya menjadi kelasmu yang sebenarnya.
3. Ubah ukuran huruf menjadi `30px` memakai `kelas.style.fontSize = "30px";`

Refleksi singkat Bagian 2: Hal yang masih membuatku bingung: perbedaan antara texcontent dan inner html

6. BAGIAN 3: MEMILIH ELEMEN DENGAN `getElementsByClassName` DAN `getElementsByTagName`

6.1 `getElementsByClassName`

Memilih **semua elemen** yang memiliki `class` tertentu.

> **Analogi:** `class` seperti **nama kelas**. Jika guru berkata "semua anak X RPL 1 berdiri!", maka **banyak siswa** berdiri sekaligus.

```
document.getElementsByClassName("namaClass");
```

> ⚠️ Perhatikan huruf **s** pada `Elements`. Karena hasilnya **banyak**, namanya jamak.

6.2 `getElementsByTagName`

Memilih **semua elemen** berdasarkan **nama tag**.

> **Analogi:** seperti memanggil "semua yang berstatus *siswa*".

```
document.getElementsByTagName("li");    // semua <li>
document.getElementsByTagName("p");     // semua <p>
```

6.3 Hasilnya Adalah Koleksi, Bukan Satu Elemen

Hasil kedua metode ini disebut **HTMLCollection**, yaitu kumpulan elemen yang mirip **daftar urut** (array). Setiap anggota punya **nomor indeks yang dimulai dari 0**.

> **Analogi:** indeks seperti **nomor lantai di lift**. Di banyak gedung, lantai paling bawah disebut lantai 0 (lantai dasar), lantai di atasnya lantai 1, dan seterusnya. Demikian juga elemen pertama dalam koleksi bernomor 0, elemen kedua bernomor 1.

Misalkan ada tiga elemen berkelas `buah`:

```
<p class="buah">Apel</p>    <!-- indeks 0 -->
<p class="buah">Jeruk</p>   <!-- indeks 1 -->
<p class="buah">Mangga</p>  <!-- indeks 2 -->

const buah = document.getElementsByClassName("buah");

buah.length;    // 3  → jumlah elemen
buah[0];        // elemen pertama (Apel)
buah[1];        // elemen kedua (Jeruk)
buah[2];        // elemen ketiga (Mangga)
```

> 🔑 **Hati-hati:** elemen pertama itu indeks **0**, bukan 1. Ini kesalahan paling sering dilakukan pemula.

Kesalahan yang sering terjadi: langsung mengubah koleksi seolah satu elemen.

```
buah.style.color = "red";       // ❌ ERROR, karena buah adalah koleksi
buah[0].style.color = "red";    // ✅ BENAR, memilih elemen indeks ke-0
```

6.4 Mengolah Semua Elemen dengan Perulangan (`for`)

Untuk mengubah **semua** elemen di koleksi, kita gunakan perulangan:

> **Analogi:** perulangan seperti **guru mengabsen satu per satu** dari nomor absen paling awal sampai nomor paling akhir. Guru tidak berhenti sampai semua nama selesai dipanggil.

```
for (let i = 0; i < buah.length; i++) {
  buah[i].style.color = "green";
}
```

Cara membacanya:

Bagian	Arti
`let i = 0`	mulai dari nomor 0
`i < buah.length`	ulangi selama `i` masih kurang dari jumlah elemen
`i++`	setelah satu putaran, tambahkan `i` dengan 1
`buah[i]`	ambil elemen ke-`i` pada putaran ini

Jalannya perulangan untuk 3 elemen:

```
Putaran 1: i = 0 → ubah buah[0]
Putaran 2: i = 1 → ubah buah[1]
Putaran 3: i = 2 → ubah buah[2]
i = 3 → 3 < 3 salah → berhenti
```

🧪 PRAKTIKUM 3: Memilih Banyak Elemen

Langkah 1. Buat folder `praktikum3` berisi `index.html` dan `script.js`.

Langkah 2. Isi `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum 3 - Class dan Tag</title>
</head>
<body>
  <h2>Daftar Buah</h2>
  <p class="buah">Apel</p>
  <p class="buah">Jeruk</p>
  <p class="buah">Mangga</p>

  <h2>Daftar Hobi</h2>
  <ul>
    <li>Ngoding</li>
    <li>Gaming</li>
    <li>Musik</li>
  </ul>

  <script src="script.js"></script>
</body>
</html>
```

Langkah 3. Isi `script.js`:

```
// ===== Bagian A: getElementsByClassName =====
const buah = document.getElementsByClassName("buah");
```

```
console.log(buah);          // lihat koleksinya
console.log(buah.length);   // jumlah elemen
console.log(buah[0]);       // elemen pertama

// mengubah elemen tertentu
buah[0].style.color = "red";

// mengubah semua elemen dengan perulangan
for (let i = 0; i < buah.length; i++) {
  buah[i].style.fontWeight = "bold";
  buah[i].textContent = (i + 1) + ". " + buah[i].textContent;
}

// ===== Bagian B: getElementsByTagName =====
const semuaLi = document.getElementsByTagName("li");

console.log(semuaLi.length);

for (let i = 0; i < semuaLi.length; i++) {
  semuaLi[i].style.backgroundColor = "lightyellow";
}
```

Langkah 4. Jalankan dan lengkapi tabel pengamatan:

Pertanyaan	Jawabanmu
Berapa nilai `buah.length`?	3
Apa yang tampil pada `console.log(buah[0])`?	
Bagaimana tampilan teks buah setelah dijalankan?	
Berapa nilai `semuaLi.length`?	3-

Langkah 5 (Eksperimen). Ubah `buah[0]` menjadi `buah[3]` pada baris `buah[3].style.color = "red";`. Apa yang terjadi di Console? Mengapa?

Muncul pesan error Uncaught TypeError: Cannot read properties of undefined (reading 'style'). Teks Apel juga tidak berubah menjadi warna merah.Karena jumlah elemen buah hanya ada 3, sehingga indeks yang tersedia hanya 0, 1, dan 2. Ketika dipanggil buah indeks ke-3, nilainya adalah undefined. Karena nilainya undefined, JavaScript tidak bisa mengubah properti style dari nilai tersebut sehingga terjadi error.

1. Ubah warna teks **elemen terakhir** pada daftar `li` menjadi biru. (Petunjuk: indeks terakhir = `semuaLi.length - 1`.)
2. Pilih semua tag `<h2>` dengan `getElementsByTagName("h2")`, lalu ubah warnanya menjadi `teal` memakai perulangan.

Refleksi singkat Bagian 3: Aku sekarang paham bahwa indeks pertama dimulai dari angka 0 karena dalam JavaScript (dan array/koleksi elemen), urutan posisi item dihitung mulai dari 0, bukan 1 -

7. BAGIAN 4: MEMILIH ELEMEN DENGAN `querySelector` DAN `querySelectorAll`

7.1 Konsep

`querySelector` dan `querySelectorAll` adalah cara **modern** dan **paling fleksibel**. Kita menulis selector **persis seperti di CSS**. Jadi apa yang sudah kamu pelajari di CSS bisa langsung dipakai.

> **Analogi:** `querySelector` seperti **mencari alamat dengan aplikasi peta**. Kita bisa mengetik alamat yang sangat spesifik, misalnya "gedung A lantai 2 ruang RPL 1" (`#gedungA .lantai2 .ruang`). `querySelector` membawamu ke **tempat pertama** yang cocok, sedangkan `querySelectorAll` menampilkan **semua tempat** yang cocok.

Metode	Hasil
`querySelector("selector")`	**satu** elemen, yaitu yang **pertama** cocok (atau `null`)
`querySelectorAll("selector")`	**semua** elemen yang cocok (disebut **NodeList**)

7.2 Ingat Kembali Selector CSS

Selector	Artinya	Contoh
`p`	semua tag `<p>`	`"p"`
`.kartu`	semua elemen class `kartu`	`".kartu"`
`#judul`	elemen dengan id `judul`	`"#judul"`
`ul li`	semua `li` **di dalam** `ul`	`"ul li"`
`.kartu .harga`	elemen class `harga` di dalam class `kartu`	`".kartu .harga"`
`li:first-child`	`li` yang menjadi anak pertama	`"li:first-child"`
`li:nth-child(2)`	`li` yang menjadi anak ke-2	`"li:nth-child(2)"`

> 🔑 **Beda penting!** Pada `getElementById("judul")` kita menulis **tanpa** `#`. Pada `querySelector("#judul")` kita **harus pakai** `#` (dan untuk class pakai `.`).

7.3 Contoh Penggunaan

```
const judul = document.querySelector("#judul");         // elemen ber-id judul
const kartu1 = document.querySelector(".kartu");        // kartu PERTAMA saja
const semuaKartu = document.querySelectorAll(".kartu"); // SEMUA kartu
```

7.4 Mengolah NodeList dengan `forEach`

`querySelectorAll` menghasilkan **NodeList**. Kamu boleh memakai `for` biasa (seperti di Bagian 3), atau memakai `forEach` yang lebih ringkas:

```
semuaKartu.forEach(function (kartu) {
  kartu.style.border = "2px solid teal";
});
```

Cara membaca: "untuk **setiap** `kartu` di dalam `semuaKartu`, lakukan hal berikut."

> **Analogi:** `forEach` seperti guru yang berkata, "**Setiap** siswa, buka buku halaman 10!" Perintahnya cukup diucapkan satu kali, tetapi dijalankan oleh semua siswa di dalam daftar.

Versi singkat (disebut *arrow function*, boleh kamu pakai jika sudah paham):

```
semuaKartu.forEach((kartu) => {
  kartu.style.border = "2px solid teal";
});
```

7.5 Rangkuman Perbedaan Semua Metode

Metode	Dicari berdasarkan	Hasil jika ada	Jika tidak ada	Penulisan
`getElementById("a")`	id	1 elemen	`null`	tanpa `#`
`getElementsByClassName("a")`	class	HTMLCollection	koleksi kosong	tanpa `.`
`getElementsByTagName("a")`	tag	HTMLCollection	koleksi kosong	tanpa `<>`
`querySelector("...")`	selector CSS	1 elemen pertama	`null`	`#a` / `.a` / `a`
`querySelectorAll("...")`	selector CSS	NodeList	NodeList kosong	`#a` / `.a` / `a`

🧪 PRAKTIKUM 4: Memilih dengan Selector CSS

Langkah 1. Buat folder `praktikum4` berisi `index.html` dan `script.js`.

Langkah 2. Isi `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum 4 - querySelector</title>
</head>
<body>
  <h1 id="judul">Toko Online RPL</h1>

  <ul class="menu">
    <li>Beranda</li>
    <li>Produk</li>
    <li>Kontak</li>
  </ul>

  <div class="kartu">
    <h3>Keyboard</h3>
    <p class="harga">Rp150.000</p>
  </div>
  <div class="kartu">
    <h3>Mouse</h3>
    <p class="harga">Rp80.000</p>
  </div>
  <div class="kartu">
    <h3>Headset</h3>
    <p class="harga">Rp200.000</p>
  </div>
```

```
  <script src="script.js"></script>
</body>
</html>
```

Langkah 3. Isi `script.js`:

```
// ===== querySelector: memilih SATU elemen =====
const judul = document.querySelector("#judul");
judul.style.color = "darkblue";

const kartuPertama = document.querySelector(".kartu");
console.log(kartuPertama);

const hargaPertama = document.querySelector(".kartu .harga");
hargaPertama.style.fontWeight = "bold";

const menuPertama = document.querySelector(".menu li:first-child");
menuPertama.style.color = "red";

// ===== querySelectorAll: memilih BANYAK elemen =====
const semuaKartu = document.querySelectorAll(".kartu");
console.log(semuaKartu.length);

semuaKartu.forEach(function (kartu) {
  kartu.style.border = "2px solid teal";
  kartu.style.padding = "10px";
  kartu.style.margin = "10px 0";
});

const semuaHarga = document.querySelectorAll(".harga");
semuaHarga.forEach((harga) => {
  harga.style.color = "green";
});

const semuaMenu = document.querySelectorAll(".menu li");
console.log(semuaMenu.length);
```

Langkah 4. Jalankan lalu lengkapi tabel:

Pertanyaan	Jawabanmu
Berapa isi `semuaKartu.length`?	`semuaKartu.forEach(function (kartu) { kartu.style.border = "2px solid teal"; kartu.style.padding = "10px"; kartu.style.margin = "10px 0"; });`
Kartu mana yang dipilih oleh `document.querySelector(".kartu")`?	`<div class="kartu"> <h3>Keyboard</h3> <p class="harga">Rp150.000</p>`
Berapa isi `semuaMenu.length`?	`<ul class="menu"> <li>Beranda</li> <li>Produk</li> <li>Kontak</li> </ul>`
Menu mana yang berwarna merah?	• Beranda

Langkah 5 (Eksperimen). Ganti `querySelector(".kartu")` menjadi `querySelector("kartu")` (tanpa titik). Apa yang tampil di Console? Mengapa?

_

```
input user > JS script4.js > ...
1   // ===== querySelector: memilih SATU elemen =====
2   const judul = document.querySelector("#judul");
3   judul.style.color = "darkblue";
4
5   // Perubahan pada Langkah 5: ".kartu" diganti menjadi "kartu" (tanpa titik)
6   const kartuPertama = document.querySelector("kartu");
7   console.log(kartuPertama); // Akan menampilkan: null
8
9   const hargaPertama = document.querySelector(".kartu .harga");
10  hargaPertama.style.fontWeight = "bold";
11
12  const menuPe  any  document.querySelector(".menu li:first-child");
13  menuPertama.style.color = "red";
14
15  // ===== querySelectorAll: memilih BANYAK elemen =====
16  const semuaKartu = document.querySelectorAll(".kartu");
17  console.log(semuaKartu.length);
18
19  semuaKartu.forEach(function (kartu) {
20    kartu.style.border = "2px solid teal";
21    kartu.style.padding = "10px";
22    kartu.style.margin = "10px 0";
23  });
24
25  const semuaHarga = document.querySelectorAll(".harga");
26  semuaHarga.forEach((harga) => {
27    harga.style.color = "green";
28  });
29
30  const semuaMenu = document.querySelectorAll(".menu li");
31  console.log(semuaMenu.length);
```

Hal ini terjadi karena tanda titik pada selector CSS digunakan untuk memilih elemen berdasarkan class, yaitu .kartu. Ketika tanda titik tersebut dihapus sehingga menjadi "kartu", JavaScript akan menganggapnya sebagai pencarian tag atau elemen HTML yang bernama `<kartu`. Karena tag `<kartu>` tidak ditemukan di dalam dokumen index.html, fungsi querySelector akhirnya mengembalikan nilai null.

🎯 **Tantangan Mandiri 4:**

1. Ubah warna **item menu ke-2** (Produk) menjadi oranye memakai `li:nth-child(2)`.
2. Ubah teks semua `<h3>` di dalam `.kartu` menjadi huruf kapital semua memakai `querySelectorAll(".kartu h3")` dan `textContent.toUpperCase()`.

Refleksi singkat Bagian 4: Menurutku metode seleksi yang paling mudah dipakai adalah Menurutku metode seleksi yang paling mudah dipakai adalah `queryselector` karena karena **penulisannya fleksibel dan sama seperti sintaks selector di CSS**.

8. BAGIAN 5: SETELAH DIPILIH, LALU APA? (MENGUBAH TEKS DAN GAYA)

Tujuan memilih elemen adalah agar kita bisa **mengubahnya**. Berikut alat-alat dasar yang akan kamu pakai terus.

8.1 Mengubah Isi Elemen

Properti	Fungsi	Contoh
`textContent`	mengubah/membaca **teks biasa**	`judul.textContent = "Halo";`
`innerHTML`	mengubah/membaca isi **termasuk tag HTML**	`judul.innerHTML = "Halo <i>kamu</i>";`

Perbedaannya:

> **Analogi:** `textContent` seperti **mesin ketik biasa**: apa pun yang diketik muncul apa adanya, termasuk tanda `<b>`. `innerHTML` seperti **Microsoft Word**: perintah format (tebal, miring) ikut dijalankan dan hasilnya langsung terlihat.

```
elemen.textContent = "<b>Tebal</b>";  // tampil apa adanya: <b>Tebal</b>
elemen.innerHTML   = "<b>Tebal</b>";  // tampil sebagai teks tebal
```

8.2 Mengubah Gaya (CSS) Elemen

Gunakan `elemen.style.namaProperti = "nilai"`.

Aturan penulisan: nama properti CSS yang memakai tanda hubung diubah menjadi **camelCase** (huruf besar di awal kata kedua).

> **Analogi:** `style` seperti **mengganti seragam atau mengecat ulang tembok**. Orang atau tembok yang sama (elemennya tidak berubah), hanya penampilannya yang diganti.

Di CSS	Di JavaScript
`color`	`style.color`
`background-color`	`style.backgroundColor`
`font-size`	`style.fontSize`
`border-radius`	`style.borderRadius`

```
elemen.style.backgroundColor = "yellow";
elemen.style.fontSize = "24px";
```

> Nilai selalu ditulis sebagai **teks** (dalam tanda kutip), lengkap dengan satuannya, misalnya `"24px"`.

8.3 Menjalankan Kode Saat Tombol Diklik (Pengantar)

Agar perubahan tidak langsung terjadi saat halaman dibuka, kita bisa membungkus kode ke dalam **fungsi**, lalu memanggilnya dari tombol:

```
<button onclick="gantiTema()">Klik Saya</button>

function gantiTema() {
  // kode yang dijalankan saat tombol diklik
}
```

Fungsi adalah **sekumpulan perintah yang diberi nama**, dan baru dijalankan ketika "dipanggil". (Materi *event* akan dipelajari lebih dalam di pertemuan berikutnya.)

> **Analogi:** fungsi seperti **resep masakan di buku**. Resep sudah tertulis, tetapi belum ada yang dimasak sebelum seseorang "memanggilnya". Tombol `onclick` adalah orang yang membuka buku dan mulai memasak.

🧪 PRAKTIKUM 5: Tombol Ganti Tema (Gabungan Semua Metode)

Langkah 1. Buat folder `praktikum5` berisi `index.html` dan `script.js`.

Langkah 2. Isi `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum 5 - Ganti Tema</title>
</head>
<body>
  <h1 id="judul">Belajar DOM Itu Seru</h1>
  <p class="isi">Paragraf pertama.</p>
  <p class="isi">Paragraf kedua.</p>

  <button onclick="gantiGelap()">Tema Gelap</button>
  <button onclick="gantiTerang()">Tema Terang</button>

  <script src="script.js"></script>
</body>
</html>
```

Langkah 3. Isi `script.js`:

```
function gantiGelap() {
  document.body.style.backgroundColor = "#222222";
  document.getElementById("judul").style.color = "gold";

  const semuaIsi = document.querySelectorAll(".isi");
  semuaIsi.forEach((p) => {
    p.style.color = "white";
  });
}

function gantiTerang() {
  document.body.style.backgroundColor = "white";
  document.getElementById("judul").style.color = "black";

  const semuaIsi = document.getElementsByClassName("isi");
  for (let i = 0; i < semuaIsi.length; i++) {
    semuaIsi[i].style.color = "black";
  }
```

```
}
```

Langkah 4. Klik kedua tombol bergantian. Perhatikan bahwa `gantiGelap` memakai `querySelectorAll`, sedangkan `gantiTerang` memakai `getElementsByClassName`, tetapi **hasil akhirnya sama**.

Pertanyaan	Jawabanmu
Elemen apa saja yang berubah saat "Tema Gelap" diklik?	tulisan “belajar doom itu seru” berubah menjadi warna kuning,tulisan di bawah nya menjadi putih karena latar berubah manjadi hitam
Mengapa hasil kedua cara memilih bisa sama?	Karena `querySelectorAll(".isi")` dan `getElementsByClassName("isi")` sama-sama mengambil **seluruh elemen** yang memiliki class `isi`. Melakukan perulangan (`forEach` atau `for`) pada hasilnya akan memberikan efek perubahan warna teks yang sama ke setiap elemen tersebut.

🎯 Tantangan Mandiri 5:

1. Tambahkan tombol ketiga "Tema Hijau" yang mengubah latar belakang menjadi `lightgreen` dan teks `.isi` menjadi `darkgreen`.
2. Tambahkan tombol "Ganti Judul" yang mengubah teks `#judul` menjadi namamu.



Refleksi singkat Bagian 5: Bagian yang paling seru menurutku: Saat membuat fungsi ganti tema dengan tombol, karena tampilan web bisa langsung berubah warna secara instan saat diklik.

9. KESALAHAN UMUM DAN CARA MEMPERBAIKINYA

Programmer profesional pun sering membuat kesalahan. Yang membedakan adalah kemampuan **membaca error dan memperbaikinya**. Pakai tabel ini sebagai pertolongan pertama:

No	Gejala / Pesan di Console	Kemungkinan Penyebab	Cara Memperbaiki
1	`Cannot read properties of null`	id/class salah ketik, atau elemen tidak ada	Cek ejaan di HTML dan JS, samakan persis
2	Sama seperti di atas, padahal ejaan benar	`<script>` diletakkan **sebelum** elemen	Pindahkan `<script>` ke sebelum `</body>`
3	`... is not a function`	Salah ketik nama metode (misal `getElementbyId`)	Cek huruf besar/kecil: `getElementById`
4	`Cannot set properties of undefined` / `style` tidak ada	Mengubah **koleksi** langsung, tanpa indeks	Gunakan `koleksi[0]` atau perulangan
5	Tidak ada yang berubah, tidak ada error	File `script.js` salah nama/lokasi, atau belum di-*refresh*	Cek `src`, simpan file, tekan `Ctrl + F5`
6	`querySelector` menghasilkan `null`	Lupa `#` atau `.`	Gunakan `"#id"` atau `".class"`
7	Elemen terakhir tidak berubah	Salah indeks atau `i <= length`	Gunakan `i < koleksi.length`
8	`Unexpected token` / `missing )`	Kurung atau tanda kutip tidak berpasangan	Periksa tanda `( )` `" "` `{ }`

Strategi Debugging (mencari kesalahan) 4 Langkah:

1. **Baca** pesan error di Console, lihat nama file dan nomor barisnya.
2. **Cek** ejaan, huruf besar-kecil, dan tanda baca pada baris tersebut.
3. **Cetak** dengan `console.log(variabel)` untuk memastikan isi variabelnya apa.
4. **Ubah satu hal**, lalu uji lagi. Jangan mengubah banyak hal sekaligus.

10. RANGKUMAN DAN GLOSARIUM

Rangkuman

1. **JavaScript** membuat halaman web menjadi interaktif.
2. **DOM** adalah representasi halaman HTML dalam bentuk pohon objek yang bisa dikendalikan JavaScript.
3. **DOM Selection** = memilih elemen sebelum mengubahnya.
4. `getElementById` memilih **1 elemen** via id (tanpa `#`).
5. `getElementsByClassName` dan `getElementsByTagName` memilih **banyak elemen** (koleksi, indeks dimulai dari **0**).
6. `querySelector` memilih **elemen pertama** yang cocok, `querySelectorAll` memilih **semua** yang cocok, memakai **selector CSS** (`#id`, `.class`, `tag`).
7. Letakkan `<script>` **sebelum** `</body>`.
8. Gunakan `textContent`, `innerHTML`, dan `style` untuk mengubah elemen yang sudah dipilih.

Peta Analogi

Ingat analogi berikut jika kamu lupa fungsi sebuah konsep:

Konsep	Analogi di kehidupan sehari-hari
HTML, CSS, JavaScript	Kerangka rumah, cat dan dekorasi, serta listrik
DOM	Silsilah keluarga: ada induk, anak, dan saudara
DOM Selection	Memanggil atau mencari orang tertentu di dalam keluarga
`getElementById`	Memanggil siswa dengan NIS (hanya satu orang)
`getElementsByClassName`	Memanggil semua siswa satu kelas (banyak orang)
`getElementsByTagName`	Memanggil semua yang berstatus "siswa"
`querySelector`	Aplikasi peta: membawa ke tempat pertama yang cocok
`querySelectorAll`	Aplikasi peta: menampilkan semua tempat yang cocok
Variabel	Kotak penyimpanan berlabel
Indeks (mulai dari 0)	Nomor lantai di lift (lantai dasar = 0)
Perulangan `for` dan `forEach`	Guru mengabsen satu per satu
`null`	Memanggil NIS yang tidak terdaftar, tidak ada yang maju
`textContent` dan `innerHTML`	Mesin ketik biasa dan Microsoft Word
`style`	Mengganti seragam atau mengecat tembok
Fungsi	Resep masakan yang baru dimasak saat dipanggil
Console	Papan skor dan buku catatan di bengkel

Cheat Sheet

```
// Memilih
document.getElementById("id")
document.getElementsByClassName("class")
document.getElementsByTagName("tag")
document.querySelector("selector")
document.querySelectorAll("selector")

// Mengubah
elemen.textContent = "teks baru";
elemen.innerHTML = "<b>HTML baru</b>";
elemen.style.color = "red";
elemen.style.backgroundColor = "yellow";

// Perulangan
for (let i = 0; i < koleksi.length; i++) { koleksi[i] ... }
nodeList.forEach((item) => { item ... });
```

Glosarium

Istilah	Arti
DOM	Document Object Model, representasi halaman web berbentuk pohon
Node	Satu titik/bagian pada pohon DOM
Elemen	Node berupa tag HTML
Selector	Aturan untuk menunjuk elemen tertentu
Variabel	Wadah bernama untuk menyimpan data/elemen
Array / Koleksi	Kumpulan data berurutan dengan nomor indeks mulai 0
HTMLCollection	Koleksi elemen hasil `getElementsBy...`
NodeList	Koleksi elemen hasil `querySelectorAll`
Indeks	Nomor urut anggota pada koleksi (dimulai dari 0)
Console	Layar pesan/error milik browser untuk programmer
null	Nilai yang berarti "tidak ada" (elemen tidak ditemukan)
Fungsi	Kumpulan perintah yang diberi nama dan bisa dipanggil